

Roteiro de Laboratório: Governança em ITIL com Python

Curso: Sistemas de Informação

Disciplina: Gestão de Sistemas de Informação

Tema: Governança de TI – ITIL aplicado em Python

Continuidade do Monitor de SLA

1. Fundamentação Teórica

1.1 SLA e Governança de TI (Revisão)

O SLA (Service Level Agreement ou Acordo de Nível de Serviço) é um compromisso ou acordo firmado entre um provedor de serviços de TI e o seu cliente. O seu principal propósito é descrever o serviço de TI oferecido, especificar as responsabilidades, obrigações e direitos de ambas as partes, e documentar as metas de nível de serviço esperadas.

Dentro do contexto da Governança de TI, o SLA atua na definição de metas mensuráveis de desempenho. Estabelecer essas metas (também chamadas de Metas de Nível de Serviço) é fundamental para garantir que a operação do serviço gere os resultados esperados pelo negócio.

A definição destas metas mensuráveis possui as seguintes características essenciais:

- **Uso de Indicadores de Desempenho:**

As metas são fundamentadas em indicadores de desempenho práticos. As metas mensuráveis devem abranger, por exemplo, horários de uso, disponibilidade, capacidade do serviço, quantidade de usuários suportados, tempo de resposta a transações, tempo de reparo e níveis de segurança.

- **Linguagem Focada no Negócio:**

As metas e o documento do SLA devem ser elaborados de forma clara, concisa e livre de jargões técnicos. O acordo não deve incluir diagramas de procedimentos complexos, mas sim métricas que as áreas de negócio consigam compreender e validar.

- **Responsabilidade Bilateral:**

A definição das metas cria uma relação de mão dupla. Para que a TI atinja as metas de desempenho medidas, o cliente também se compromete a cumprir certas regras de utilização do serviço.

As metas mensuráveis de desempenho no SLA servem como uma ferramenta para alinhar as expectativas do cliente à real capacidade de entrega da TI, assegurando que o projeto do serviço esteja sempre adequado ao seu propósito final.

Neste laboratório, daremos continuidade à aplicação prática de SLA e Governança de TI, ampliando o foco para três processos centrais do ITIL:

- Gerenciamento de Disponibilidade;
- Gerenciamento de Mudanças (Change Management);
- Gerenciamento de Incidentes com priorização por fila.

1.2 Gerenciamento de Disponibilidade (Availability Management)

O Gerenciamento de Disponibilidade é o processo ITIL responsável por garantir que os serviços de TI estejam disponíveis sempre que o negócio precisar deles, conforme acordado no SLA. A disponibilidade é normalmente expressa como um percentual de tempo em que o serviço esteve operacional em relação ao tempo total acordado de operação.

A fórmula básica é:

$$\text{Disponibilidade (\%)} = ((\text{Tempo Acordado} - \text{Tempo de Indisponibilidade}) / \text{Tempo Acordado}) \times 100$$

Exemplo: para um mês de 30 dias, o tempo total acordado é de 43.200 minutos. Se um sistema ficou 200 minutos indisponível, sua disponibilidade foi:

$$\text{Disponibilidade} = ((43.200 - 200) / 43.200) \times 100$$

Metas típicas de mercado variam, por exemplo, entre 99,0%, 99,5% ou 99,9% de disponibilidade mensal. Sempre que o valor real ficar abaixo da meta do SLA, há uma **violação de disponibilidade**, que deve ser registrada, analisada e reportada.

1.3 Gerenciamento de Mudanças (Change Management)

O Gerenciamento de Mudanças controla o ciclo de vida de todas as mudanças realizadas na infraestrutura e nos serviços de TI, com o objetivo de minimizar riscos e impactos negativos ao negócio. Toda mudança deve ser registrada como uma **RFC (Request for Change)**, classificada e aprovada antes de ser implementada.

Tipos comuns de mudança no ITIL:

- **Mudança Padrão:**

Mudanças de baixo risco, rotineiras, previamente aprovadas, com procedimento bem definido (por exemplo, criação de contas de usuário em sistema).

- **Mudança Normal:**

Mudanças que exigem avaliação de impacto e aprovação formal, podendo passar por comitê de mudanças (CAB).

- **Mudança de Emergência:**

Mudanças que precisam ser implementadas rapidamente para corrigir um incidente grave ou evitar grande impacto. Exigem controle ainda mais rigoroso de registro e justificativa.

Violações de governança nesse contexto incluem:

- Mudanças implementadas **sem aprovação prévia**;
- Mudanças de emergência não registradas ou sem justificativa;
- Excesso de mudanças em curto período, elevando o risco de indisponibilidade.

1.4 Gerenciamento de Incidentes com Priorizações por Fila

O Gerenciamento de Incidentes tem como objetivo restaurar a operação normal do serviço o mais rápido possível após uma interrupção (incidente). Uma boa prática ITIL é organizar o atendimento em **filas de suporte**, como:

- Fila Nível 1 (Service Desk / Atendimento Inicial);
- Fila Nível 2 (Suporte Técnico Especializado);
- Fila Nível 3 (Fornecedores / Desenvolvimento / Terceiros).

Para cada fila, podem existir **metas de tempo de atendimento** diferentes, conforme a criticidade dos incidentes. Por exemplo:

- Incidentes críticos na Fila Nível 1 devem ser atendidos em até 15 minutos;
- Incidentes críticos na Fila Nível 2 devem ser solucionados em até 2 horas;
- Incidentes de baixa prioridade podem ter prazos maiores.

Neste laboratório, iremos utilizar Python para:

- Monitorar disponibilidade de serviços;
- Auditar o processo de mudanças;
- Avaliar o tempo de espera e de resolução em diferentes filas de atendimento.

2. Laboratório Prático – Continuidade do Monitor de SLA

Você desenvolverá scripts em Python para auditar indicadores de Governança de TI com base em registros fictícios da empresa TechStore S.A., utilizando as bibliotecas **pandas** e, quando necessário, **matplotlib**.

Atividade 1 – Monitor de Disponibilidade de Serviços

Objetivo: Calcular a disponibilidade mensal de serviços de TI e identificar violações da meta de disponibilidade definida em SLA.

Cenário:

A TechStore S.A. opera quatro serviços críticos. O SLA estabelece uma meta mínima de **99,5% de disponibilidade mensal** para cada um deles. O time de operações registrou o tempo de indisponibilidade (em minutos) ao longo de um mês de 30 dias (43.200 minutos).

Tarefa:

Criar um script em Python que:

1. Calcule a disponibilidade real (%) de cada serviço;
2. Identifique as violações da meta de disponibilidade;
3. Gere um relatório de auditoria;
4. Produza um gráfico de barras comparando a disponibilidade real com a meta do SLA.

Abaixo, um código-base que pode ser adaptado:

```
import pandas as pd
import matplotlib.pyplot as plt

#
=====
====
# ATIVIDADE 1 – GERENCIAMENTO DE DISPONIBILIDADE (ITIL)
#
=====
====

# 1. Configuração do período analisado
minutos_no_mes = 43200 # 30 dias * 24 horas * 60
minutos

# 2. Log simplificado de indisponibilidades dos serviços
dados_disponibilidade = {
    'servico': ['ERP', 'Portal do Cliente', 'Sistema de
Pagamentos', 'E-mail Corporativo'],
    'tempo_indisponivel_min': [80, 250, 15, 420]
}

df_disp = pd.DataFrame(dados_disponibilidade)
```

```

# 3. Meta de disponibilidade definida no SLA
meta_disponibilidade = 99.5 # em %

# --- INÍCIO DA TAREFA DO ALUNO ---

# 4. Calcule a disponibilidade real (%) de cada serviço
df_disp['disponibilidade_pct'] = round(
    ((minutos_no_mes -
df_disp['tempo_indisponivel_min']) / minutos_no_mes) *
    100,
    3
)

# 5. Identifique as violações (serviços com
disponibilidade abaixo da meta)
violacoes_disp = df_disp[df_disp['disponibilidade_pct']
< meta_disponibilidade]

# 6. Exiba o relatório de auditoria de disponibilidade
print("=== RELATÓRIO DE DISPONIBILIDADE – TECHSTORE S.A.
===")
print(df_disp)

print("\n" + "=" * 60)
print(f"VIOLAÇÕES DE DISPONIBILIDADE (Meta:
{meta_disponibilidade}%)")
print("=" * 60)

if violacoes_disp.empty:
    print("Nenhuma violação detectada. Todos os serviços
dentro da meta.")
else:
    print(violacoes_disp)
    print(f"\nTotal de serviços em não-conformidade:
{len(violacoes_disp)}")

# 7 (Opcional). Gere um gráfico de barras com a
disponibilidade por serviço
cores = [
    'red' if d < meta_disponibilidade else 'steelblue'
    for d in df_disp['disponibilidade_pct']
]

plt.figure(figsize=(9, 4))
plt.bar(df_disp['servico'],
df_disp['disponibilidade_pct'], color=cores)

```

```
plt.axhline(y=meta_disponibilidade, color='orange',
            linestyle='--',
            label=f'Meta SLA ({meta_disponibilidade}%)')
plt.title('Disponibilidade Mensal dos Serviços –
TechStore S.A.')
plt.xlabel('Serviço')
plt.ylabel('Disponibilidade (%)')
plt.ylim(98.5, 100.1)
plt.legend()
plt.tight_layout()
plt.show()

# --- FIM DA TAREFA DO ALUNO ---
```

Atividade 2 – Auditoria do Gerenciamento de Mudanças

Objetivo: Identificar mudanças não autorizadas (violação de governança) e analisar a distribuição de tipos de mudança.

Cenário:

O processo de Change Management da TechStore S.A. exige que toda mudança em produção possua aprovação registrada antes da implementação. Foi gerado um log de RFCs (Requests for Change) do último mês, incluindo tipo de mudança, status de aprovação e informação se a mudança foi implementada.

Tarefa:

Criar um script em Python que:

1. Identifique mudanças implementadas sem aprovação (não autorizadas);
2. Calcule a taxa de não-conformidade;
3. Mostre a distribuição dos tipos de mudança;
4. Gere um gráfico de pizza com a proporção de cada tipo.

Código-base sugerido:

```
import pandas as pd
import random
import matplotlib.pyplot as plt
```

```

#
=====
====
# ATIVIDADE 2 – GERENCIAMENTO DE MUDANÇAS (ITIL)
#
=====
====

random.seed(7)

# 1. Geração do log de RFCs (Requests for Change)
n_mudancas = 25
tipos = ['Normal', 'Padrao', 'Emergencia']
status_aprovacao = ['Aprovada', 'Aprovada', 'Aprovada',
'Nao Aprovada'] # ~25% sem aprovação

dados_mudancas = {
    'rfc_id': [f'RFC-{i}' for i in range(3001, 3001 +
n_mudancas)],
    'tipo': [random.choice(tipos) for _ in
range(n_mudancas)],
    'aprovacao': [random.choice(status_aprovacao) for _
in range(n_mudancas)],
    'implementada': ['Sim'] * n_mudancas # Todas foram
para produção
}

df_mudancas = pd.DataFrame(dados_mudancas)

# --- INÍCIO DA TAREFA DO ALUNO ---

# 2. Identifique mudanças implementadas sem aprovação
(violação de governança)
nao_autorizadas = df_mudancas[
    (df_mudancas['implementada'] == 'Sim') &
    (df_mudancas['aprovacao'] == 'Nao Aprovada')
]

print("=== LOG DE MUDANÇAS – TECHSTORE S.A. ===")
print(df_mudancas)

print("\n" + "=" * 55)
print("AUDITORIA DE CHANGE MANAGEMENT")
print("Mudanças implementadas SEM aprovação prévia:")
print("=" * 55)

if nao_autorizadas.empty:

```

```

        print("Nenhuma não-conformidade detectada.")
    else:
        print( nao_autorizadas)
        print(f"\nTotal de mudanças não autorizadas:
        {len(nao_autorizadas)}")
        taxa = round((len(nao_autorizadas) /
        len(df_mudancas)) * 100, 1)
        print(f"Taxa de não-conformidade: {taxa}%")

# 3. Distribuição por tipo de mudança
contagem_tipos = df_mudancas['tipo'].value_counts()
print("\nDistribuição de mudanças por tipo:")
print(contagem_tipos)

# 4 (Opcional). Gráfico de pizza dos tipos de mudança
plt.figure(figsize=(6, 6))
plt.pie(
    contagem_tipos,
    labels=contagem_tipos.index,
    autopct='%1.1f%%',
    startangle=140,
    colors=['steelblue', 'mediumseagreen', 'tomato']
)
plt.title('Distribuição de Mudanças por Tipo – TechStore
S.A.')
plt.tight_layout()
plt.show()

# --- FIM DA TAREFA DO ALUNO ---

```

Atividade 3 – Gerenciamento de Incidentes com Priorização por Fila

Objetivo: Simular o atendimento de incidentes em diferentes filas (Nível 1 e Nível 2) e identificar violações de SLA tanto no tempo de espera quanto no tempo total de resolução para incidentes de alta prioridade.

Cenário:

A TechStore S.A. estruturou seu Service Desk em duas filas principais:

- **Fila Nível 1 (N1):** atendimento inicial, triagem e resolução de incidentes mais simples;

- **Fila Nível 2 (N2):** atendimento especializado, responsável pelos incidentes mais complexos.

A empresa definiu o seguinte SLA para incidentes de prioridade Alta:

- **Tempo máximo de espera na Fila Nível 1:** 15 minutos (0,25 horas);
- **Tempo total máximo de resolução (N1 + N2, se houver):** 2 horas.

Um log fictício de incidentes foi gerado com as seguintes informações:

- `ticket_id` – identificador do incidente;
- `prioridade` – Alta, Média ou Baixa;
- `fila_inicial` – fila que recebeu o incidente primeiro (N1 ou N2);
- `tempo_espera_horas` – tempo em horas até o início do atendimento na fila inicial;
- `tempo_resolucao_total_horas` – tempo total (da abertura até a resolução final).

Tarefa:

Criar um script em Python que:

1. Gere um log fictício de incidentes com filas;
 2. Aplique as metas de SLA para incidentes de prioridade Alta;
 3. Identifique:
 - Incidentes de prioridade Alta que violaram o SLA de tempo de espera em N1;
 - Incidentes de prioridade Alta que violaram o SLA de tempo total de resolução;
1. Gere um relatório de governança para a gestão de filas de incidentes.

Código-base sugerido:

```

import pandas as pd
import random

#
=====
====
# ATIVIDADE 3 – GERENCIAMENTO DE INCIDENTES E FILAS
(ITIL)
#
=====
=====

random.seed(123)

# 1. Configuração do log de incidentes
n_incidentes = 20
prioridades = ['Alta', 'Media', 'Baixa']
filas_iniciais = ['N1', 'N2']

dados_incidentes = {
    'ticket_id': range(2001, 2001 + n_incidentes),
    'prioridade': [random.choice(prioridades) for _ in
range(n_incidentes)],
    'fila_inicial': [random.choice(filas_iniciais) for _
in range(n_incidentes)],
    # tempo de espera entre 0 e 0.8h (0 a 48 minutos)
    'tempo_espera_horas': [round(random.uniform(0.0,
0.8), 2) for _ in range(n_incidentes)],
    # tempo total entre 0.5h e 5h
    'tempo_resolucao_total_horas':
[round(random.uniform(0.5, 5.0), 2) for _ in
range(n_incidentes)]
}

df_incidentes = pd.DataFrame(dados_incidentes)

# 2. Metas de SLA da TechStore S.A. para incidentes de
prioridade Alta
meta_espera_n1_alta = 0.25      # 15 minutos
meta_resolucao_alta = 2.0       # 2 horas

# --- INÍCIO DA TAREFA DO ALUNO ---

print("=== LOG DE INCIDENTES POR FILA – TECHSTORE S.A.
===")
print(df_incidentes)

```

```

# 3. Filtrar apenas incidentes de prioridade Alta
incidentes_alta =
df_incidentes[df_incidentes['prioridade'] == 'Alta']

# 4. Violações de SLA de tempo de espera (apenas quando
fila inicial é N1)
violacoes_espera_n1 = incidentes_alta[
    (incidentes_alta['fila_inicial'] == 'N1') &
    (incidentes_alta['tempo_espera_horas'] >
meta_espera_n1_alta)
]

# 5. Violações de SLA de tempo total de resolução
(independente da fila)
violacoes_resolucao_total = incidentes_alta[
    incidentes_alta['tempo_resolucao_total_horas'] >
meta_resolucao_alta
]

print("\n" + "=" * 65)
print("RELATÓRIO DE GOVERNANÇA - GERENCIAMENTO DE
INCIDENTES (PRIORIDADE ALTA)")
print("=" * 65)

# 6. Exibição das violações de tempo de espera em N1
print("\nA) Violações de SLA de TEMPO DE ESPERA na Fila
N1 (Alta):")
print(f"Critério: espera > {meta_espera_n1_alta} horas
(15 minutos)")

if violacoes_espera_n1.empty:
    print("Nenhuma violação de tempo de espera em N1
para incidentes de prioridade Alta.")
else:
    print(violacoes_espera_n1)
    print(f"Total de violações de espera em N1:
{len(violacoes_espera_n1)}")

# 7. Exibição das violações de tempo total de resolução
print("\nB) Violações de SLA de TEMPO TOTAL DE RESOLUÇÃO
(Alta):")
print(f"Critério: resolução total >
{meta_resolucao_alta} horas")

if violacoes_resolucao_total.empty:
    print("Nenhuma violação de tempo total de resolução
para incidentes de prioridade Alta.")

```

```
else:
    print(violacoes_resolucao_total)
    print(f"Total de violações de tempo total de
    resolução: {len(violacoes_resolucao_total)}")

# 8. Estatísticas gerais (opcional)
print("\nResumo de incidentes de prioridade Alta:")
print(incidentes_alta[['tempo_espera_horas',
'tempo_resolucao_total_horas']].describe())

# --- FIM DA TAREFA DO ALUNO ---
```